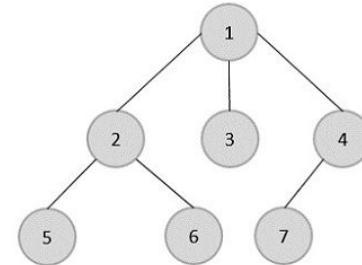


Trees and Heaps

Data Structures

What is a tree?

A tree is a **hierarchical** data structure used to represent data in a **parent-child** relationship.



General Tree Data Structure

What is a tree?

A connected, acyclic (no cycles) graph with:

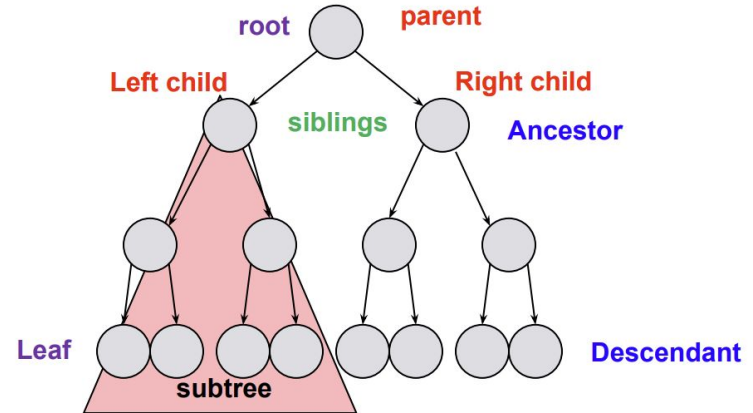
- A **root** node that has 0 or more subtrees
- Exactly one path between any two nodes

Binary tree

- Tree where each node has at most 2 children

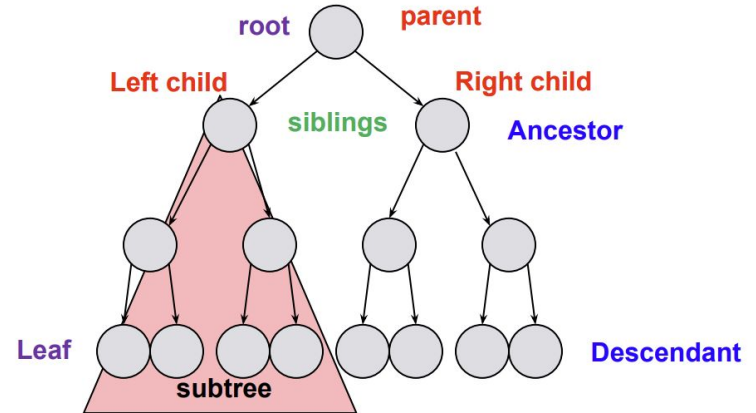
d-ary tree

- Tree where each node has at most d children



Tree Definitions

- **Parent(i)**: Node directly above node i
- **Child(i)**: Node directly below node i
- **Siblings**: Children of the same parent
- **Root**: Only node with no parent
- **Leaf**: Node with 0 children
- **Subtree(n)**: Tree rooted at node n



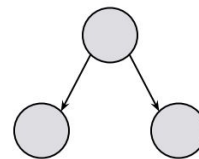
Tree Definitions

Full d-ary tree, T, where

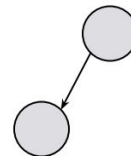
- Every vertex has 0 or d children and all leaf nodes are at the same level (i.e. adding 1 more node requires increasing the height of the tree)

Complete d-ary tree

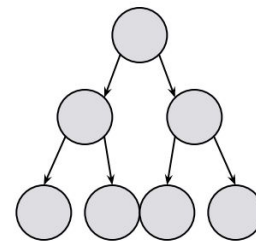
- Top h-1 levels are full AND bottom level is filled left-to-right
- Each level is filled left-to-right and a new level is not started until the previous one is complete



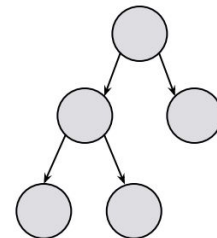
Full



Complete, but not full



Full



Complete, but not full

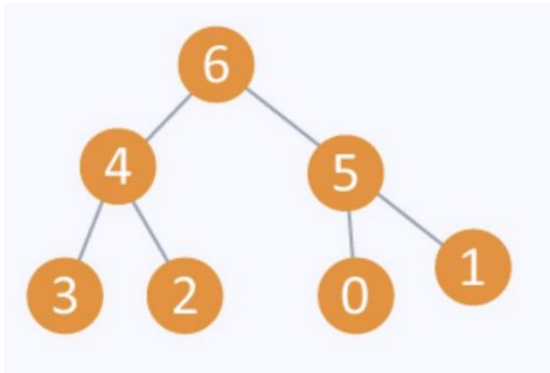
What is a heap?

Heap Property

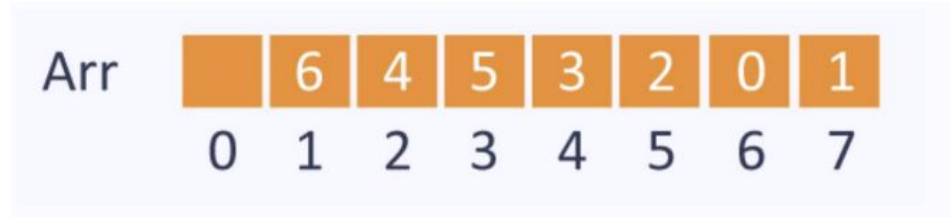
A **tree** where every parent is better than its children

Two representations:

As a complete d-ary tree



As an array



What is a heap?

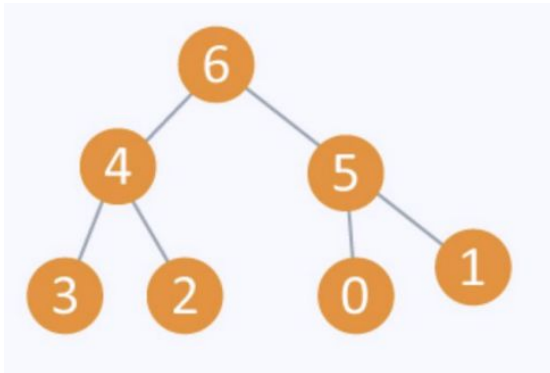
Heap Property

A **tree** where every parent is better than its children

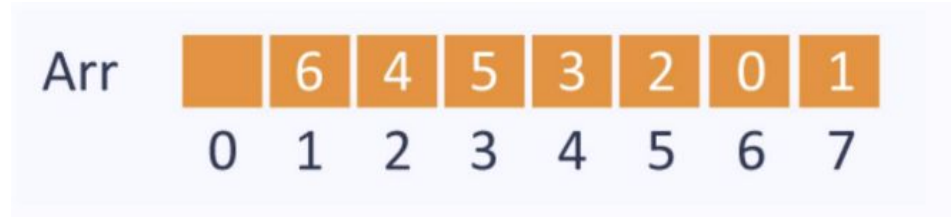
Note: Binary tree that is complete can be stored nicely in an array

Two representations:

As a complete d-ary tree



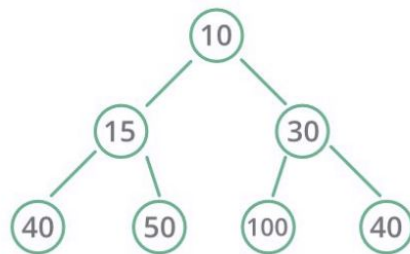
As an array



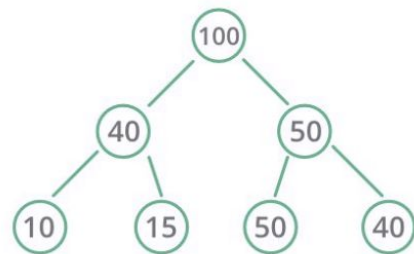
Heap property

- **Min Heap:** node is less than or equal to all its children
- **Max Heap:** node is greater than or equal to all its children

Heap Data Structure



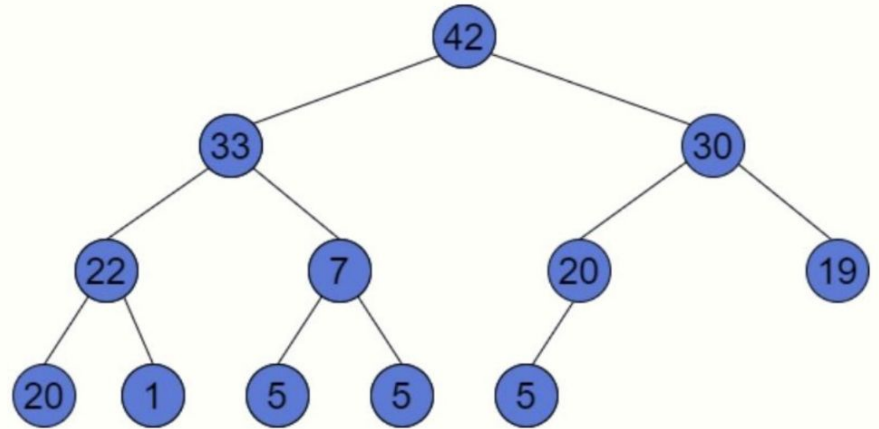
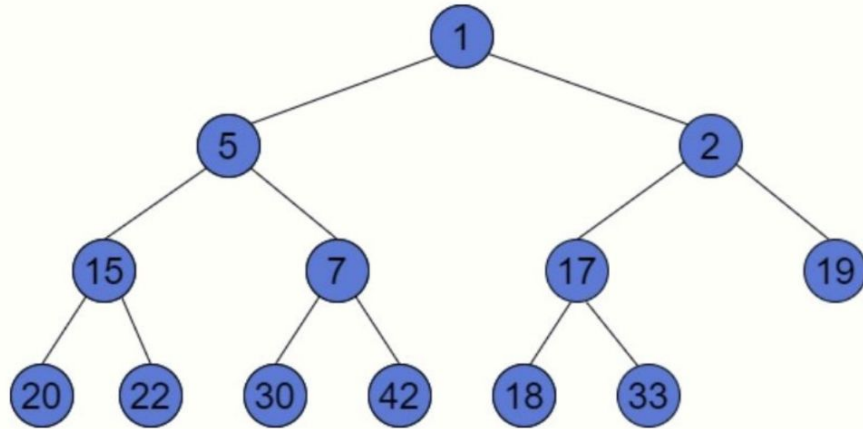
Min Heap



Max Heap

Question

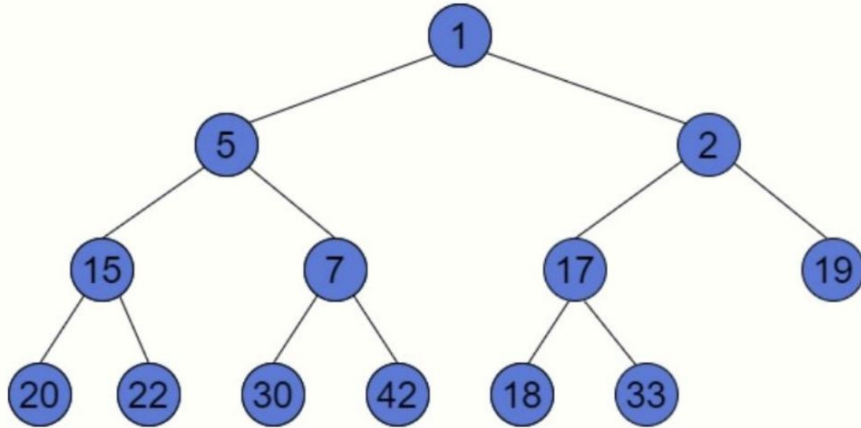
Which one is a min heap and which one is a max heap?



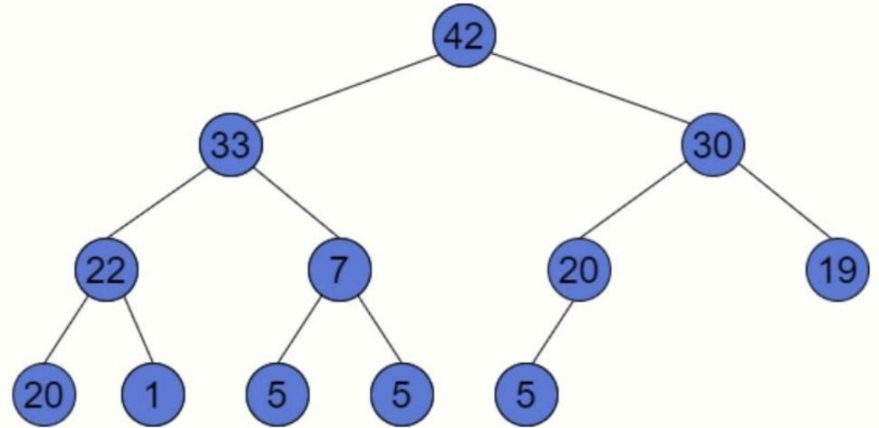
Question

Which one is a min heap and which one is a max heap?

Min Heap



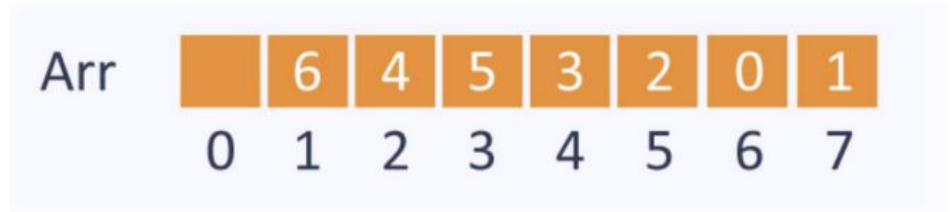
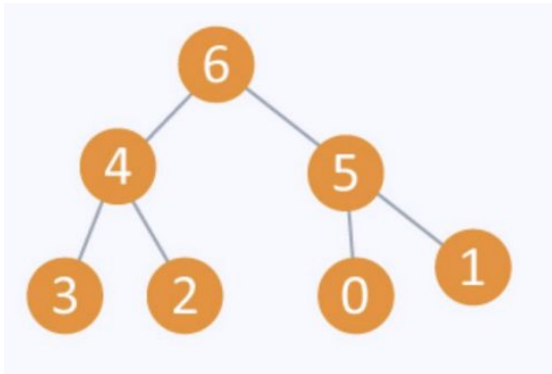
Max Heap



Store heaps in array

Mathematical relation for finding the index of node i 's parent, left, and right child?

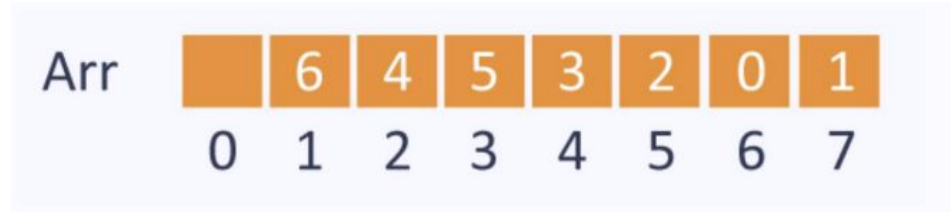
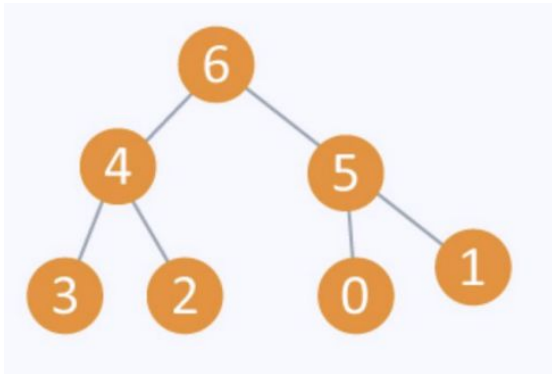
- Let's say it starts at index 1 and index 0 is empty



Store heaps in array

Mathematical relation for finding the index of node i 's parent, left, and right child?

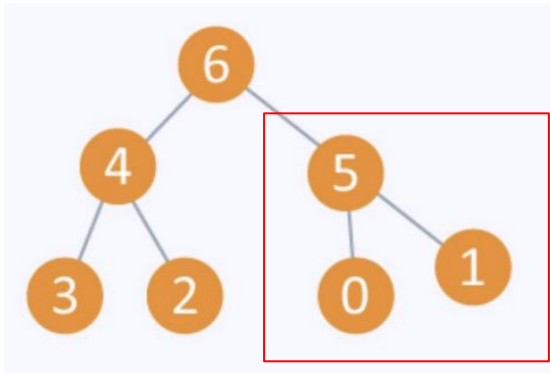
- Let's say it starts at index 1 and index 0 is empty
- $\text{Parent}(i) = i/2$
- $\text{Left_child}(i) = 2*i$
- $\text{Right_child}(i) = 2*i + 1$



Store heaps in array

- $\text{Parent}(i) = i/2$
- $\text{Left_child}(i) = 2*i$
- $\text{Right_child}(i) = 2*i + 1$

- Index of 5: **3**
- $\text{Parent}(5) = 3/2 = 1$
- $\text{Left_child}(5) = 2*3 = 6$
- $\text{Right_child}(5) = 2*3+1 = 7$

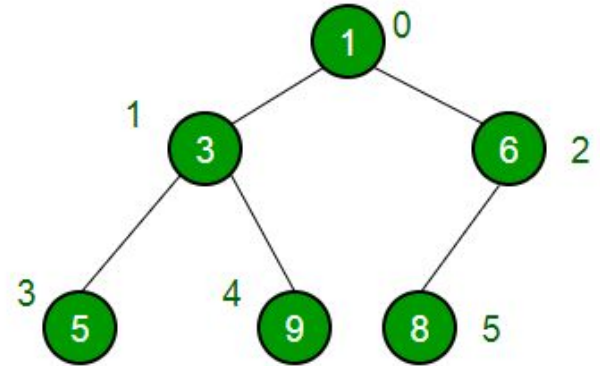


Store heaps in array (start with index 0)

Mathematical relation for the index of node i's parent, left, and right child?

Let's say it starts at index 0

- $\text{Parent}(i) = (i-1)/2$
- $\text{Left_child}(i) = 2i+1$
- $\text{Right_child}(i) = 2i+2$

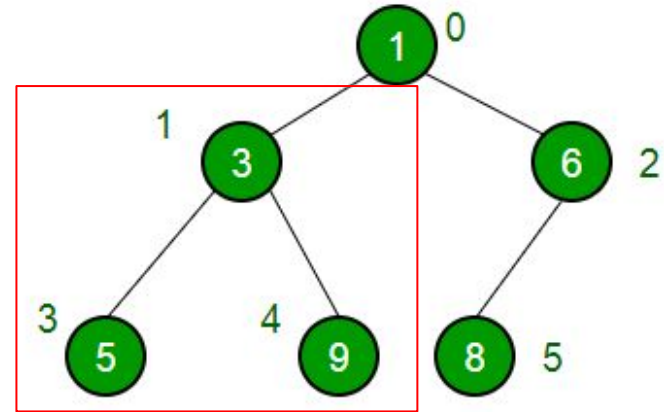


1	3	6	5	9	8
0	1	2	3	4	5

Store heaps in array (start with index 0)

- $\text{Parent}(i) = (i-1)/2$
- $\text{Left_child}(i) = 2i+1$
- $\text{Right_child}(i) = 2i+2$

- Index of 3: **1**
- $\text{Parent}(3) = 1/2 = 0$
- $\text{Left_child}(3) = 2*1 + 1 = 3$
- $\text{Right_child}(3) = 2*1+2 = 4$



1	3	6	5	9	8
0	1	2	3	4	5

Next Lecture

- Heap operations: push / pop an element
- Code implementations: buildHeap(), push(), pop()
- Priority queues
- Complexity analysis